

CO1-AT-3- SCENARIO BASED QUESTIONS

Name:B.Nikhila

Register Name: 192472102

Course Code: DSA0404

Course Name: Fundamentals of Data Science

- 1. Scenario:** You are working on a project that involves analyzing a dataset containing information about houses in a neighborhood. The dataset is stored in a CSV file, and you have imported it into a NumPy array named `house_data`. Each row of the array represents a house, and the columns contain various features such as the number of bedrooms, square footage, and sale price.

Question: Using NumPy arrays and operations, how would you find the average sale price of houses with more than four bedrooms in the neighborhood?

House_ID	Bedrooms	Sqft	Sale_Price
H1	3	1400	7500000
H2	5	2200	12500000
H3	4	1800	9800000
H4	6	3000	18500000
H5	2	1100	5200000

Ans:

Code:

```
import numpy as np
house_data = np.array([
    [3, 1400, 7500000],
    [5, 2200, 12500000],
    [4, 1800, 9800000],
    [6, 3000, 18500000],
    [2, 1100, 5200000]
])
filtered = house_data[house_data[:, 0] > 4]
average_price = np.mean(filtered[:, 2])
print("Average Sale Price:", average_price)
```

```
Scenario-01.py - C:/Users/sekha/OneDrive/Desktop/DSA0404-FODS/Scenario-01.py (3.13.8)
File Edit Format Run Options Window Help

import numpy as np
house_data = np.array([
    [3, 1400, 7500000],
    [5, 2200, 12500000],
    [4, 1800, 9800000],
    [6, 3000, 18500000],
    [2, 1100, 5200000]
])
filtered = house_data[house_data[:, 0] > 4]
average_price = np.mean(filtered[:, 2])
print("Average Sale Price:", average_price)

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:al5ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
==== RESTART: C:/Users/sekha/OneDrive/Desktop/DSA0404-FODS/Scenario-01.py ====
Average Sale Price: 15500000.0
>>>
```

2. **Scenario:** An email service provider wants to automatically classify incoming emails as spam or non-spam. Each email is represented using features such as number of links, number of capitalized words, and total email length. Historical email data is labeled as spam (1) or not spam (0).

Question:

Develop a Logistic Regression model using the given features.

Predict whether a newly received email is spam or not.

Email_ID	Num_Links	Num_Caps_Words	Email_Length	Spam
E1	1	3	120	0
E2	6	15	300	1
E3	0	1	80	0
E4	4	10	250	1
E5	2	5	160	0

Ans:

Code:

```
import numpy as np
```

```
from sklearn.linear_model import LogisticRegression
```

```
X = np.array([
```

```
    [1, 3, 120],
```

```
    [6, 15, 300],
```

```
    [0, 1, 80],
```

```
    [4, 10, 250],
```

```
    [2, 5, 160]
```

```
])
```

```
y = np.array([0, 1, 0, 1, 0])
```

```
model = LogisticRegression()
```

```
model.fit(X, y)
```

```
links = int(input("Enter Number of Links: "))
```

```
caps = int(input("Enter Number of Capitalized Words: "))
```

```
length = int(input("Enter Email Length: "))
```

```
new_email = np.array([[links, caps, length]])
```

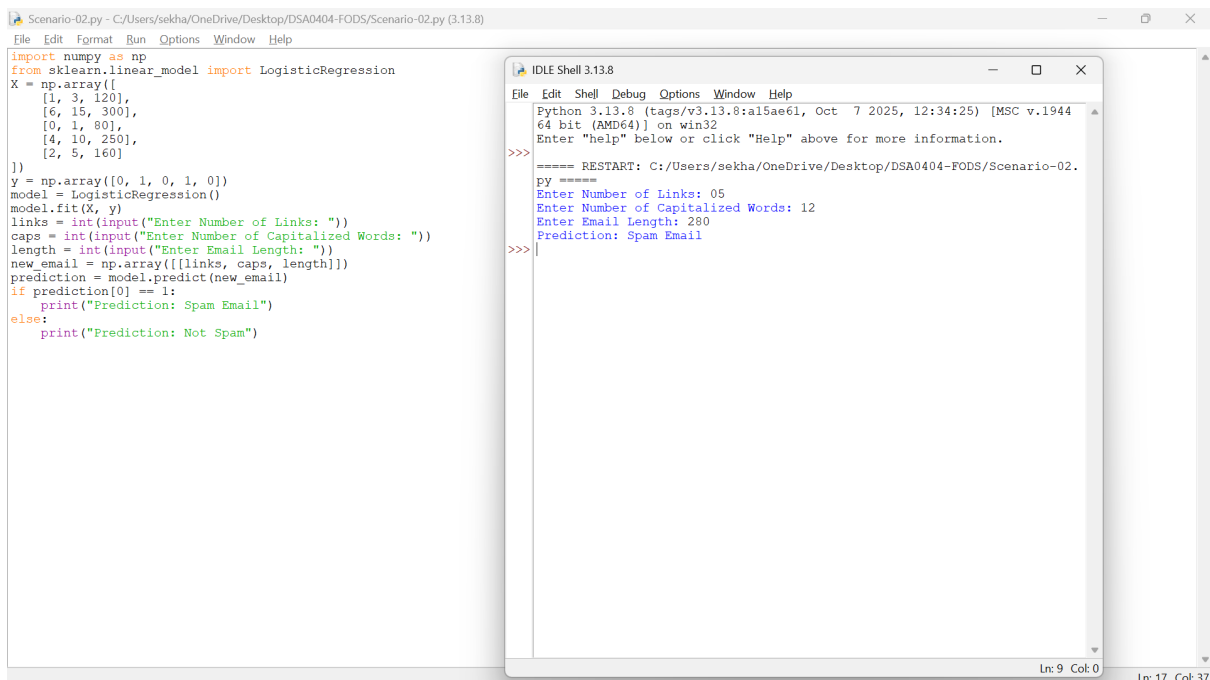
```
prediction = model.predict(new_email)
```

```
if prediction[0] == 1:
```

```
    print("Prediction: Spam Email")
```

```
else:
```

```
    print("Prediction: Not Spam")
```



The screenshot shows a Python IDE with two windows. The main window displays the code from the previous blocks, with syntax highlighting. The second window, titled 'IDLE Shell 3.13.8', shows the execution output. It starts with a restart message, followed by the user's inputs: 05 for links, 12 for capitalized words, and 280 for email length. The final output is 'Prediction: Spam Email'.

```
Scenario-02.py - C:/Users/sekha/OneDrive/Desktop/DSA0404-FODS/Scenario-02.py (3.13.8)
File Edit Format Run Options Window Help
import numpy as np
from sklearn.linear_model import LogisticRegression
X = np.array([
    [1, 3, 120],
    [6, 15, 300],
    [0, 1, 80],
    [4, 10, 250],
    [2, 5, 160]
])
y = np.array([0, 1, 0, 1, 0])
model = LogisticRegression()
model.fit(X, y)
links = int(input("Enter Number of Links: "))
caps = int(input("Enter Number of Capitalized Words: "))
length = int(input("Enter Email Length: "))
new_email = np.array([[links, caps, length]])
prediction = model.predict(new_email)
if prediction[0] == 1:
    print("Prediction: Spam Email")
else:
    print("Prediction: Not Spam")

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:al5a661, Oct 7 2025, 12:34:25) [MSC v.1944
64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/sekha/OneDrive/Desktop/DSA0404-FODS/Scenario-02.
py =====
Enter Number of Links: 05
Enter Number of Capitalized Words: 12
Enter Email Length: 280
Prediction: Spam Email
>>>
```

Ln: 9 Col: 0
Ln: 17 Col: 37

- 3. Scenario:** An email service provider wants to automatically classify incoming emails as spam or non-spam. Each email is represented using features such as number of links, number of capitalized words, and total email length. Historical email data is labeled as spam (1) or not spam (0).

Question:

Develop a Logistic Regression model using the given features.

Predict whether a newly received email is spam or not.

Email_ID	Num_Links	Num_Caps_Words	Email_Length	Spam
E1	1	3	120	0
E2	6	15	300	1
E3	0	1	80	0
E4	4	10	250	1
E5	2	5	160	0

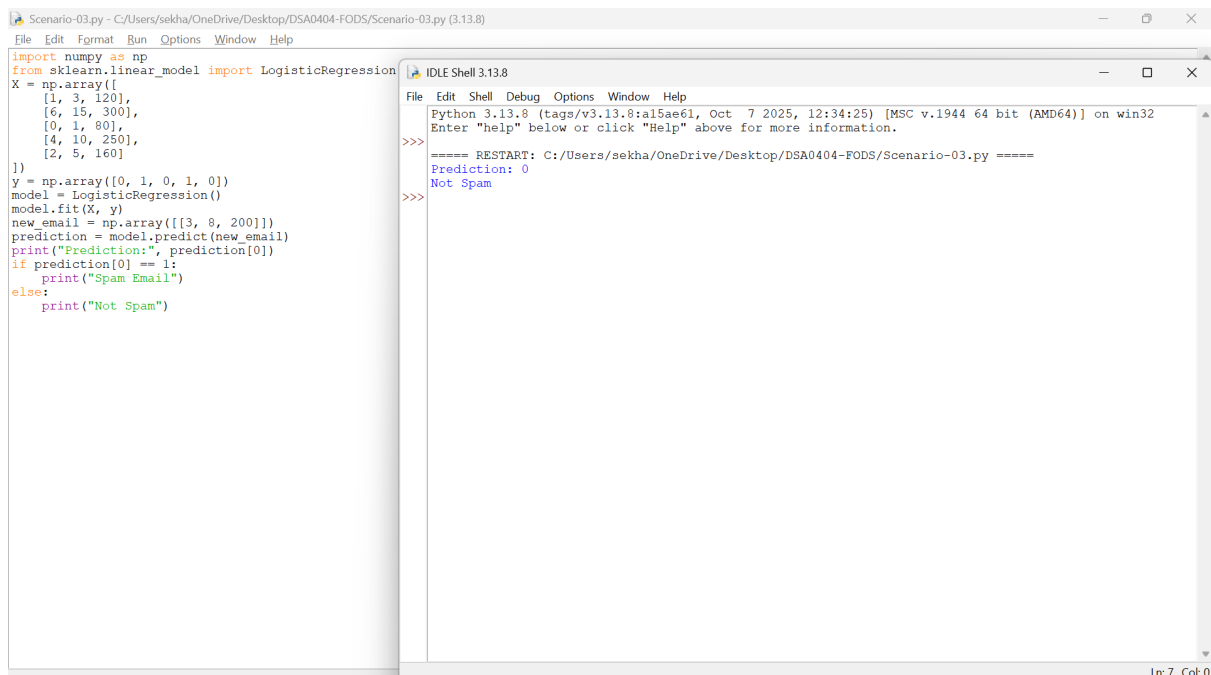
Ans:

Code:

```
import numpy as np
from sklearn.linear_model import LogisticRegression
X = np.array([
    [1, 3, 120],
    [6, 15, 300],
    [0, 1, 80],
    [4, 10, 250],
    [2, 5, 160]
])
y = np.array([0, 1, 0, 1, 0])
model = LogisticRegression()
model.fit(X, y)
new_email = np.array([[3, 8, 200]])
prediction = model.predict(new_email)
print("Prediction:", prediction[0])
if prediction[0] == 1:
    print("Spam Email")
```

else:

```
print("Not Spam")
```



The screenshot shows a Python IDE with two windows. The left window, titled 'Scenario-03.py', contains the following code:

```
import numpy as np
from sklearn.linear_model import LogisticRegression

X = np.array([
    [1, 3, 120],
    [6, 15, 300],
    [0, 1, 80],
    [4, 10, 250],
    [2, 5, 160]
])
y = np.array([0, 1, 0, 1, 0])
model = LogisticRegression()
model.fit(X, y)
new_email = np.array([[3, 8, 200]])
prediction = model.predict(new_email)
print("Prediction:", prediction[0])
if prediction[0] == 1:
    print("Spam Email")
else:
    print("Not Spam")
```

The right window, titled 'IDLE Shell 3.13.8', shows the output of the program:

```
Python 3.13.8 (tags/v3.13.8:al5ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/sekha/OneDrive/Desktop/DSA0404-FODS/Scenario-03.py =====
Prediction: 0
Not Spam
>>>
```

4. Scenario: Linear Regression for Car Resale Price Prediction You are a data analyst at a used-car marketplace. You want to predict the resale price of a car based on features such as car age (years), total kilometers driven, and engine capacity (cc). You have a dataset containing past car listings with these features and their final selling prices.

Question: Write a Python program that loads the dataset containing Age, Kms_Driven, Engine_CC, and Resale_Price, trains a Linear Regression model using scikit-learn, accepts user input for a new car, and predicts the estimated resale price.

Car_ID	Age (Years)	Kms_Driven	Engine_CC	Resale_Price
C1	2	25000	1200	650000
C2	5	60000	1500	420000
C3	3	40000	1300	520000
C4	7	90000	1800	300000
C5	1	15000	1000	720000

Ans:

Code:

```

import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LinearRegression

data = {
    "Brand": ["Maruti", "Hyundai", "Honda", "Toyota", "Tata"],
    "Kms_Driven": [25000, 60000, 40000, 90000, 15000],
    "Engine_CC": [1200, 1500, 1300, 1800, 1000],
    "Resale_Price": [650000, 420000, 520000, 300000, 720000]
}

df = pd.DataFrame(data)
label_encoder = LabelEncoder()
df["Brand"] = label_encoder.fit_transform(df["Brand"])
X = df[["Brand", "Kms_Driven", "Engine_CC"]]
y = df["Resale_Price"]
model = LinearRegression()
model.fit(X, y)
print("Available Brands:")
brands = label_encoder.classes_
for i, brand in enumerate(brands):
    print(i, "-", brand)

brand_name = input("\nEnter Brand Name: ")
kms = int(input("Enter Kilometers Driven: "))
engine = int(input("Enter Engine CC: "))
brand_code = label_encoder.transform([brand_name])[0]
new_car = [[brand_code, kms, engine]]
predicted_price = model.predict(new_car)
print("\nEstimated Resale Price = ₹", round(predicted_price[0], 2))

```

```

Scenario-04.py - C:\Users\sekha\OneDrive\Desktop\DSA0404-FODS\Scenario-04.py (3.13.8)
File Edit Format Run Options Window Help

import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LinearRegression

data = {
    "Brand": ["Maruti", "Hyundai", "Honda", "Toyota", "Tata"],
    "Kms_Driven": [25000, 60000, 40000, 90000, 15000],
    "Engine_CC": [1200, 1500, 1300, 1800, 1000],
    "Resale_Price": [650000, 420000, 520000, 300000, 720000]
}

df = pd.DataFrame(data)
label_encoder = LabelEncoder()
df["Brand"] = label_encoder.fit_transform(df["Brand"])
X = df[["Brand", "Kms_Driven", "Engine_CC"]]
y = df["Resale_Price"]
model = LinearRegression()
model.fit(X, y)
print("Available Brands:")
brands = label_encoder.classes_
for i, brand in enumerate(brands):
    print(i, "-", brand)
brand_name = input("\nEnter Brand Name: ")
kms = int(input("Enter Kilometers Driven: "))
engine = int(input("Enter Engine CC: "))
brand_code = label_encoder.transform([brand_name])[0]
new_car = [[brand_code, kms, engine]]
predicted_price = model.predict(new_car)
print("\nEstimated Resale Price = ₹", round(predicted_price[0], 2))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:1a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:\Users\sekha\OneDrive\Desktop\DSA0404-FODS\Scenario-04.py =====
>>>
Available Brands:
0 - Honda
1 - Hyundai
2 - Maruti
3 - Tata
4 - Toyota

Enter Brand Name: Toyota
Enter Kilometers Driven: 45000
Enter Engine CC: 1300

Warning (from warnings module):
  File "C:\Users\sekha\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\utils\validation.py", line 2691
    warnings.warn(
UserWarning: X does not have valid feature names, but LinearRegression was fitted with feature names

Estimated Resale Price = ₹ 561702.41
>>>
Ln: 22 Col: 0

```

5. Scenario: Loan Approval (CART Classification): A bank wants to decide whether to approve or reject a loan application. The decision depends on features like income, credit score, and existing loans. Past applications are labeled as Approved (1) or Rejected (0).

Question: Using CART, build a Decision Tree Classifier for loan approval.

Predict the class for a new applicant and display the decision rules.

Applicant_ID	Income (₹/Month)	Credit_Score	Existing_Loans	Approved
A1	45000	720	0	1
A2	30000	650	1	0
A3	60000	780	0	1
A4	28000	620	2	0
A5	52000	710	1	1

Ans:

Code:

```

import numpy as np
from sklearn.tree import DecisionTreeClassifier, export_text

X = np.array([
    [45000, 720, 0],
    [30000, 650, 1],

```

```

[60000, 780, 0],
[28000, 620, 2],
[52000, 710, 1]
])
y = np.array([1, 0, 1, 0, 1])
model = DecisionTreeClassifier(criterion="gini", random_state=42)
model.fit(X, y)
income = int(input("Enter Monthly Income: "))
credit = int(input("Enter Credit Score: "))
loans = int(input("Enter Existing Loans: "))
new_applicant = np.array([[income, credit, loans]])
prediction = model.predict(new_applicant)
print("\nPrediction Result")
if prediction[0] == 1:
    print("Loan Approved")
else:
    print("Loan Rejected")
feature_names = ["Income", "Credit_Score", "Existing_Loans"]
rules = export_text(model, feature_names=feature_names)
print("\nDecision Tree Rules:")
print(rules)

```

The screenshot shows a Python IDE with two windows. The main window displays the code from the previous block. The IDLE Shell window shows the execution output:

```

Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/sekha/OneDrive/Desktop/DSA0404-FODS/Scenario-05.py =====
Enter Monthly Income: 50000
Enter Credit Score: 530
Enter Existing Loans: 0

Prediction Result
Loan Rejected

Decision Tree Rules:
|---- Credit_Score <= 680.00
| |---- class: 0
|---- Credit_Score > 680.00
| |---- class: 1
>>>

```

Ln: 18 Col: 0

Ln: 27 Col: 0